

Finance forwards a PDF four weeks later. Thirty-day feedback loops are bad for engineering discipline.

They take instructions from text you do not control: System prompt, retrieved documents and user input get glued into one string and handed to a model that cannot tell which part carries your authority. A PDF uploaded by a customer becomes executable in a limited but real sense. Not shellcode, but it can change what your software does next.

Everything goes through one door

Most teams end up building the same component and putting it in the middle. Model gateway, AI proxy, inference router -- the name varies. Applications stop calling models directly and call the gateway instead.

This looks like unnecessary indirection until you try to enforce a spending limit across four teams that do not have one. It is the only place in the architecture where enforcement is possible.

We use LiteLLM on most engagements. It speaks the OpenAI dialect that every SDK understands and routes to whatever sits behind it, including your own vLLM boxes or an Ollama instance. Envoy AI Gateway and the Kong AI plugins work well if you already run those at the edge. Writing your own thin proxy is possible, though the open source options are mature enough that it mostly buys you maintenance work.

```
yaml
model_list:
  - model_name: fast-tier
    litellm_params:
      model: openai/qwen2.5-7b-instruct
      api_base: http://vllm-qwen.
inference.svc.cluster.local:8000/v1
      api_key: os.environ/INTERNAL_KEY

  - model_name: heavy-tier
    litellm_params:
```

```
      model: anthropic/claude-sonnet-4-6
      api_key: os.environ/ANTHROPIC_KEY

litellm_settings:
  callbacks: ["otel", "langfuse"]
  cache: true
  cache_params:
    type: redis
    host: redis.platform.svc.cluster.
local
  ttl: 3600

general_settings:
  master_key: os.environ/LITELLM_MASTER_
KEY
  max_budget: 250000
  budget_duration: 30d
  alerting: ["slack"]
```

Routing, caching, tracing hooks, budget and alerts are in one file. The platform team owns it, and product teams do not think about it.

The value shows up later. When a better open model lands next quarter, you edit a mapping. We did that for a client in Ahmedabad on a Tuesday morning: 40 minutes, no downtime, and nobody outside the platform team noticed. Without a gateway, fourteen services would each have carried their own client library and hardcoded model string in a config file nobody had opened since the engineer who wrote it left in 2024.

Traces first

This is the cheapest fix and it makes everything else visible.

OpenTelemetry now carries semantic conventions for generative AI. Model name, token counts, temperature, finish reason, and tool calls all have agreed attribute names. LLM spans land in the same Tempo or Jaeger you already run, on the same trace ID as the HTTP request that started things. An LLM call is a span, nested in the request span, next to your Postgres spans. Not a separate system with its own dashboard.

Instrumentation is cheap with OpenLLMetry or the native OTel

libraries.

```
python
from traceloop.sdk import Traceloop
from traceloop.sdk.decorators import workflow

Traceloop.init(
    app_name="claims-assistant",
    api_endpoint="http://otel-collector.
platform:4318",
)

@workflow(name="summarise_claim")
def summarise_claim(claim_id: str,
document: str):
    context = vector_store.
search(document, top_k=5)
    return llm.invoke(
        system=SUMMARY_PROMPT,
        user=f"{context}\n\n{document}",
    )
```

Spans are not the whole picture, because the unit you want to reason about is a conversation. Langfuse handles that for us. Open source -- self-hosts on Postgres and ClickHouse -- gives session grouping, prompt versioning and cost per user. It also gives a human somewhere to mark bad outputs as bad, which is how you build a test set that reflects reality.

Arize Phoenix is worth knowing for retrieval problems. If you are trying to work out why a RAG pipeline keeps pulling the wrong chunk, it will get you there faster. I use both.

```
yaml
services:
  langfuse:
    image: langfuse/langfuse:latest
    depends_on: [postgres, clickhouse]
    ports:
      - "3000:3000"
    environment:
      DATABASE_URL: postgresql://
postgres:${PG_PASS}@postgres:5432/
langfuse
      CLICKHOUSE_URL: http://
clickhouse:8123
```